Kevin Küster

Zusammenfassung: Rechnersysteme

Rechnersysteme

Finite State Machines (FSM)

Protokoll-Instanz-Automaten (PIA)

PIA: $M = (S, \Sigma, \delta, S_0)$
- ↳ S: Zustände
- ↳ Σ: Eingabealphabet ⇒ $In \cup Out \cup Hid$
- ↳ δ: Übergangsfunktion
- ↳ S_0: Startzustand

Konvention: „tx" = Empfangen
„-x" = Senden
„x" = internes Ergebnis

Eigenschaften:
- ▷ Vollständigkeit: Verhalten für alle Zustände und Eingaben definiert
- ▷ Verklemmungsfreiheit: (Deadlock-Freiheit) Keine unerwünschten Zustände ohne Folgezustände
- ▷ Garantierter Fortschritt: (Liveblock-Freiheit) Keine Schleifen ohne Fortschritt
- ▷ Terminierung: Endzustand wird immer erreicht
- ▷ Beschränktheit der Kanäle (bei Kanalkopplung)

Kopplung

- ▷ Direkte Kopplung (Synchron):
  - ↳ Regel 1: Interne Ereignisse jederzeit (nur ein Partner ändert Zustand)
  - ↳ Regel 2: Sende-/Empfangsereignisse nicht isoliert möglich
  - ↳ Regel 3: Senden + Empfangen gleichzeitig → Internes Ergebnis im Produktautomaten

  ⇒ Zeitliche Verzögerung wird ignoriert → Vereinfachung
- ▷ Kanalkopplung (asynchron, FIFO-Kanäle):
  - ↳ Regel 1: Interne Ergebnisse wie bei direkter Kopplung; Kanalinhalte unberührt
  - ↳ Regel 2: Senden → Signal wird am Ende des Kanals eingefügt; nur Sender ändert Zustand
  - ↳ Regel 3: Empfangen → Signal vom Kopf des Kanals entnommen; nur Empfänger ändert Zustand
- ▷ Zustand des Produktautomaten: Quadrupel $(Z_1 | Kanal\,1 \rightarrow 2 | Z_2)$
- ▷ Erkennbare Fehler bei Kanalkopplung: (bleiben bei direkter Kopplung ggf. verborgen)
  - ↳ Unspezifizierte Ankünfte; Deadlocks: Keine Terminierung; Keine Beschränktheit der Kanäle

Statecharts

⇒ Erweiterte Zustandsmaschinen:
- ▷ Einfache Zustände
- ▷ Zusammengesetzte Zustände: sequentielle oder nebenläufige (concurrent) Unterzustände
- ▷ Interne Aktionen: entry/exit/do-Aktivitäten
- ▷ Format: event[guard]/action-expression
- ▷ Weitere Features: Fork/Join (Nebenläufigkeit, Entscheidungen (Decision), Historie(H), Ein-/Austrittspunkte, Terminator

Timed Automata (TA)

Grundlagen TA

Timed Automata erweitern FSMs um Zeitaspekte:
- ↳ Diskrete Zustände sind „locations"
- ↳ Menge von Clocks (reellwertig), die verstrichene Zeiten messen
- ↳ Alle Clocks laufen gleich schnell
- ↳ Timing Constrains: Nur natürliche Zahlen in Vergleichen erlaubt

Komponenten einer Transition:
- ↳ Action (Aktionslabel, z.B. switch_on)
- ↳ Clock Guard / Constraint (z.B. $x \geq 100$)
- ↳ Clock Reset (z.B. reset{x})

Locations können Clock Invariants haben (z.B. „$x \leq 200$")
→ Automat muss location verlassen, bevor Invariante verletzt wird

Komposition von TA

Parallele Komposition: $A1 \parallel_{\Sigma} A_2$ (Synchronisation über gemeinsames Aktionsalphabet Σ)
- ▷ Aktionen in Σ: Müssen gleichzeitig in beiden Automaten ausgeführt werden (Handshaking)
- ▷ Aktionen nicht in Σ: Können unabhängig voneinander ausgeführt werden (Interleaving)

- ▷ Clocks bleiben ihren jeweiligen Automaten zugeordnet
- ▷ Guards und Resets werden konjunktiv kombiniert

Beispiel: Beschrankter Bahnübergang - drei interagierende TAs (Zug, Schranke, Controller)

Petrinetze (PN)

Grundbegriffe (PN)

Graphische und mathematische Beschreibungstechnik zur Modellierung von Nebenläufigkeit und Synchronisation in verteilten Systemen

Bestandteile:

Stellen (Places)	Repräsentieren Zustände, Objekte oder Variablen. Gezeichnet als Kreise
Marken (Tokens)	Befinden sich in den Stellen. Repräsentieren konkrete Objekte oder Werte. Gezeichnet als gefüllte Punkte oder Zahlen in den Kreisen
Transitionen	Modellieren Aktivitäten oder Ereignisse, die den Zustand verändern. Gezeichnet als Rechtecke
Gerichtete Kanten (Arcs)	Verbinden Stellen mit Transitionen oder umgekehrt. Nie: Stelle → Stelle, Transition → Transition
Kanten-gewichte	Optional: Zahl an Kante → Standard = 1

$PN = (P, T, I^-, I^+, M_0)$

↳ P: Menge der Stellen

↳ T: Menge der Transitionen, $P \cap T = \emptyset$

↳ I^-: Rückwärts-Inzidenzfunktion; $(p \in P, t \in T) \Rightarrow$ Kantengewicht $p \to t$
⇒ gibt Anzahl entfernter Tokens aus p beim Schalten von t

↳ I^+: Vorwärts-Inzidenzfunktion; $(p \in P, t \in T) \Rightarrow$ Anzahl eingefügter Tokens in p beim Schalten von t

↳ M_0: Anfangsmarkierung; Vektor der für jede Stelle die initiale Tokenzahl angibt: $M_0 = (1,0,0,0)^T \Rightarrow$ Stelle 1 = 1, Rest = 0

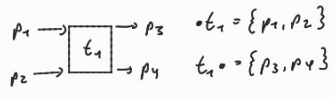
▷ $\bullet t$: Eingabestellen von t = Alle Stellen p mit $I^-(p,t) > 0$ (Kante geht zu t)

▷ $t \bullet$: Ausgabestellen von t = Alle Stellen p: $I^+(p,t) > 0$ (Kante kommt von t)

▷ $\bullet p$: Eingangstransitionen von p (Transitionen, deren Schalten Token in p legt)

▷ $p \bullet$: Ausgangstransitionen von p (Transitionen, die Tokens aus p entfernen)

Beispiel



$\bullet t_1 = \{p_1, p_2\}$

$t_1 \bullet = \{p_3, p_4\}$

Schaltregel

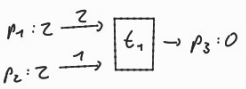
Definiert dynamisches Verhalten eines Petrinetzes

▷ Aktivierung (Enabled): Transition t aktiviert in Markierung M, wenn jede Eingangsstelle genug Token hat: $\forall p \in \bullet t: M(p) \geq I(p,t)$

▷ Schalten (Feuern, Firing): Wenn t aktiviert ist und schaltet ergibt sich neue Markierung M': $\forall p \in P: M'(p) = M(p) - I^-(p,t) + I^+(p,t)$

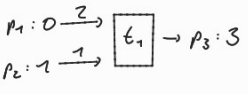
Beispiel

Vor Schalten:



$M = (2,2,0)^T$

Nach Schalten:



$M' = (0,1,3)$

Inzidenzmatrix

Inzidenzmatrix C fasst Netto-Effekt jeder Transition zusammen
$\Rightarrow C = C^+ - C^-$

↳ C^+: Vorwärts-Inzidenzmatrix; Einträge $C^+(p,t) = I^+(p,t)$

↳ C^-: Rückwärts-Inzidenzmatrix; Einträge $C^-(p,t) = I^-(p,t)$

↳ $C(p,t)$: Netto-Veränderung der Token in p, wenn t feuert

Dimension: $|P|$ Zeilen × $|T|$ Spalten

Schaltvektor

Schaltvektor f zählt für jede Transition, wie oft sie geschaltet hat:
$M = M_0 + C \cdot f$, $f \in \mathbb{N}^m$ ($m = |T|$: Anzahl Transitionen)

⇒ Einzelschritt → Einheitsvektor (1 an Position t, 0 sonst)

Beispiel

$|P| = 4$, $|T| = 5$

$$C = \begin{pmatrix} 1 & -1 & -1 & 0 & 1 \\ -2 & 2 & 0 & -2 & 0 \\ 0 & 0 & 1 & 1 & -1 \\ 0 & 0 & 1 & 1 & -1 \end{pmatrix}, \quad M_0 = (1,0,0,0)^T$$

(i) Schalten von t_2: $M' = M_0 + C \cdot e_{-\{t_2\}}$
$= (1,0,0,0)^T + (-1,2,0,0)^T = (0,2,0,0)^T$

(ii) Schalten von t_4: $M'' = M' + C \cdot e_{-\{t_4\}}$
$= (0,2,0,0)^T + (0,-2,1,1)^T = (0,0,1,1)^T$

⇒ Sequenz: $t_2\, t_4$ ⇒ $M'' = M_0 + C \cdot (0,1,0,1,0)^T$
$= (1,0,0,0)^T + (-1,0,1,1)^T = (0,0,1,1)^T$

Analyse-Eigenschaften: Beschränktheit, Lebendigkeit, Verklemmungsfreiheit

▷ Beschränktheit: Petrinetz ist k-beschränkt, wenn es $k \in \mathbb{N}$ gibt: Für jede erreichbare Markierung M und jede Stelle p gilt: $M(p) \leq k$
⇒ 1-beschränkt = sicher (safe)

Beschränktheit prüfen:
(i) Erreichbarkeitsgraph (RG) aufstellen
(ii) Für jede Stelle p das Maximum der Tokenzahlen über alle erreichbaren Markierungen bestimmen
(iii) Wenn es ein endliches k gibt, das alle Maxima begrenzt
⇒ k-beschränkt
Wenn Tokenzahl unbegrenzt wächst (RG unendlich)
⇒ Unbeschränkt

⇒ Beschränkt ⇔ endlich viele Markierungen

▷ Lebendigkeit (Liveness): Petrinetz lebendig, wenn jede Transition t von jeder erreichbaren Markierung M aus wieder aktiviert werden kann

▷ Verklemmungsfrei (Deadlock-Freiheit): Petrinetz verklemmungsfrei, wenn in jeder erreichbaren Markierung mindestens eine Transition aktiviert ist

⇒ lebendig ⇒ Verklemmungsfrei, Verklemmungsfrei ⇏ lebendig

Erreichbarkeitsgraph (RG)

Knoten: Erreichbare Markierungen
Kanten: Gefeuerte Transitionen (beschriftet mit Transitionsnamen)
Bei beschränkten Netzen: RG endlich
Bei unbeschränkten Netzen: ω-Markierung

Invariantenanalyse: P-Invariante & T-Invariante

Algebraische Methode die Eigenschaften liefert, die für jede Markierung gelten

▷ P-Invariante: Vektor $v \in \mathbb{Z}^n$, $v \neq 0$ P-Invariante, falls: $v^T \cdot C = 0$

↳ v: Spaltenvektor mit $n = |P|$

$\Rightarrow v^T \cdot M = v^T \cdot M_0$ für alle erreichbaren M;
Summe der Tokens bleibt konstant unabhängig von Schaltung

Beispiel

$$C = \begin{pmatrix} 1 & -1 & -1 & 0 & 1 \\ -2 & 2 & 0 & -2 & 0 \\ 0 & 0 & 1 & 1 & -1 \\ 0 & 0 & 1 & 1 & -1 \end{pmatrix}, \quad v = (2,1,0,2)^T$$

$\Rightarrow v^T \cdot C = 2 \cdot (1,-1,-1,0,1) + (-2,2,0,-2,0) + 2 \cdot (0,0,1,1,-1)$

$= (0,0,0,0,0)$

$\Rightarrow v^T \cdot M_0 = (2,1,0,2) \cdot (1,0,0,0)^T = 2$

↳ Für jedes erreichbare M: $2 \cdot M(p_1) + M(p_2) + 2 \cdot M(p_4) = 2$

▷ T-Invariante: Vektor $w \in \mathbb{Z}^m$, $w \neq 0$ T-Invariante gdw. $C \cdot w = 0$

$\Rightarrow$ w ist Schaltsequenz (Anzahl pro Transition), die zur Ausgangsmarkierung zurückführt

Erreichbarkeitsanalyse

Berechnet alle vom Startzustand erreichbaren Markierungen, Verwendet:

↳ Stack: Speichert noch zu untersuchende Zustände → Tiefensuche

↳ Hashtable: Speichert bereits gefundene Zustände → Duplikaterkennung

```
Init:
  push(M0, Stack)
  Insert(M0, HT)

While Stack nicht Leer DO
  state := pop(Stack)
  FOR EACH Transition t die in Stack aktiviert DO
    new_state := Fire(t, state)
    IF NOT Lookup(new_State, HT) THEN
      Insert(new_state, HT)
      push(new_State, Stack)
    END IF
  END FOR
END WHILE
```

SDL & MSC

Specification and Description language (SDL)

$\Rightarrow$ System wird beschrieben als eine Menge kommunizierender Prozesse

Geeignet für: parallel arbeitende Systeme, Telekommunikationssysteme, Echtzeitsysteme, reaktive ereignisgesteuerte Systeme

Hierarchie: System → Block → Prozess

System	Oberste Ebene - Alles was modelliert wird, inkl. Umgebung
Block	Strukturierungseinheit - kann andere Blöcke oder Prozesse enthalten ↳ Container
Prozess	Innerste Ebene - konkreter Zustandsautomat (erweiterte FSM)
Channel	Verbindung zwischen Blöcken / zur Umgebung; transportiert Signale (mit Signallisten beschriftet)
Signal	Diskrete Nachricht, die zwischen Prozessen ausgetauscht wird

▷ Blockdiagramm: Zeigt Struktur - Blöcke, Kanäle zwischen Blöcken und zur Umgebung, Signallisten an Kanälen

▷ Prozessdiagramm: Zeigt Verhalten eines Prozesses - als Flussdiagramm mit SDL-Symbolen

Achteck (abgerundetes Sechseck)	Zustand (state) - Prozess wartet auf ein gehende Signale
Pfeil mit eingehender Kerbe	INPUT (Signalempfang aus Eingangswarteschleife)
Pfeil mit ausgehender Spitze	OUTPUT (Signalversand)
Raute	ENTSCHEIDUNG - Verzweigung anhand einer Bedingung
Rechteck	TASK - Berechnung, Variablen-Zuweisung, Beliebige Aktion
Sanduhr	TIMER - Set / Reset einer Zeitbedingung
Gestricheltes Rechteck	PROZEDURAUFRUF (sub-routine-call)
Kreis mit Pfeil heraus	START
Kreis mit „X"	STOP
Save-Symbol	Signal wird zurückgestellt (nicht konsumiert → bleibt in Queue)

SDL-Prozess - Signalverarbeitung

Jeder SDL Prozess hat FIFO-Eingangswarteschleife (Queue)

Zustand SDL-Prozess:

(i) State: Eigentlicher im Prozessdiagramm gezeichneter Zustand

(ii) Variablenwerte: Aktueller Wert lokaler Variablen

(iii) Inhalt Queue: Zu verarbeitender Rest

- Verarbeitung Kopfelement Queue: Mögliche Behandlungen:
  - (i) input (Konsum): Wenn akt. Zustand Input-Transition für Signal hat; Signal wird konsumiert → Transition wird durchgeführt
  - (ii) discard (Verwerfen - implicit transition): Wenn aktueller Zustand kein passendes Input hat → Signal wird verworfen; Zustand unverändert; Signal wird aus Queue entfernt
  - (iii) save (Zurückstellen): Wenn save-Symbol für Signal im akt. Zustand vorhanden ist → Signal bleibt in der Queue, wird nicht konsumiert; Verarbeitung auf nächstes Signal
- Input-Formen:
  - (i) Asterisk-Input (*): Wild-Card, fängt jedes Signal, das nicht spezifischer behandelt wird
  - (ii) Continuous Signal: Transition wird durch erfülltsein einer boolschen Bed. ausgelöst (kein Signal nötig); Normales Signal-Input hat immer Vorrang vor continuous Signal
  - (iii) Priority-Input: Bevorzugt einen Signaltyp, auch wenn dieser nicht Kopfelement der Queue ist
- Timer-Mechanismus: Timer in SDL erzeugt nach Ablauf Timer-Signal in eigener Eingangswarteschlange → Am Ende der Queue

Message Sequenz Chart (MSC)

MSC zeigt graphisch den Nachrichtenfluss zwischen Prozessen über Zeit

Vertikale Linien	Lebenslinien der Prozesse - Eine Linie pro Prozess
Horizontale Pfeile	Nachrichten: Sender $\xrightarrow{\text{Signal-name}}$ Empfänger
Zeit-Achse	Von oben n. unten
Pfeil leicht schräg	Modelliert Übertragungszeit

Einsatz: Dokumentation von Szenarien, Test Cases

High-Level MSC (HMSC)

Zeigt das Verhalten eines Systems; Jeder Knoten ist einzelnes MSC; Kanten sind Übergänge zwischen Szenarien: Verzweigungen → alternative Szenarien

Alternativ: Inline-Ausdrücke

alt	Alternative - Auswahl zwischen Szenarien
par	Parallel - Mehrere Szenarien gleichzeitig
loop <n,m>	Schleife - n bis m Durchläufe
opt	Optional
exc	Exception

non-local Choice

Entscheidungsproblem an einer Verzweigung im HMSC wenn mehrere Prozesse beteiligt sind - Entscheidung hängt an mehreren Prozessen, aber kein gemeinsames Wissen

=> Handshakes: Vor Verzweigung wird durch zusätzliche Nachricht abgestimmt welcher Zweig gewählt wird

VHDL

VHDL - VHSIC (Very High Speed Integrated Circuit) Hardware Description Language

=> Beschreiben von Hardware-Strukturen / Algorithmischen Verhalten; Modulare Beschreibung, Hierarchiebildung

Streng Typbasiert:

- entity: Schnittstellenbeschreibung eines Bausteins (Black Box) Definiert Ports: Ein- & Ausgänge
- architecture: Realisierung eines Bausteins; Drei Strichrichtungen

↳ Funktional / Verhalten (behavioral): process-Blöcke

Beispiel

```
architecture arch_behav
of compare is
begin
  P1: process (A,B)
  begin
    if (A = B) then [-];
    else [-];
    end if;
  end process;
end arch_behav
```

↳ Datenfluss (data flow): Direkte Logik-Zuweisung (and/or/xor)

Beispiel

```
architecture arch_dataflow
of full_adder is
  signal w_1, w_2, w_3 : std_logic;
begin
  w_1 <= i_bit1 xor i_bit2;
  w_2 <= w_1 and i_carry;
[-]
```

↳ Strukturell (structual): Netzliste aus instanziierten Komponenten

Signale vs. Variablen

- Signal: <= (Pfeil)
  - ↳ Deklaration: signal name : std_logic
  - ↳ Zuweisung: name <= wert after Zeitangabe
  - => Global sichtbar innerhalb der Architecture → Nächster Wert in nächster Signalaktualisierungsphase des Simulationszyklus
- Variablen := (Doppelpunkt-Gleich)
  - ↳ Deklaration: variable name : std_logic (innerhalb Prozess)
  - ↳ Zuweisung: name := wert

Nur innerhalb eines Prozesses sichtbar → Wert sofort wirksam

VHDL - Simulationszyklen

(i) Initialisierung: Alle Signale auf Anfangswerte: Alle Prozesse einmal ausführen → bis erstes Wait-Statement

(ii) Signalaktualisierung: Aus Transaktionslisten werden Werte entnommen ↳ aktualisierung

(iii) Prozessausführung: Alle Prozesse, deren Sensitivitäts-Signale sich geändert haben (oder Wait-Bedingung erfüllt) ausführen

(iv) Zeitfortschritt: Wenn keine gültige Transaktion ansteht ↳ Simulationszeit springt zur nächsten geplanten Transaktion

Strukturelle Architektur

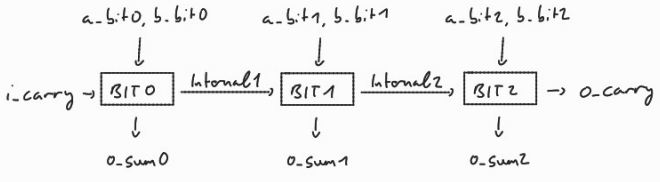
Beschreibt einen Baustein als Netzliste aus instanziierten Komponenten

- Component-Deklaration: Spiegelt Entity der zu instanziierenden Komponente wieder
- Single-Deklaration: Für interne Verbindungen zwischen Instanzen
- Instanziierung → mit eindeutigem Label und port map

Beispiel

```
architecture rtl1 of Adder3 is
 --1) Komponenten-Deklaration
 component full_adder
   port (
     i_bit1, i_bit2, i_carry : in std_logic;
     o_sum, o_carry          : out std_logic
   );
 end component;
 --2) Interne Signale für Carry-Kette
 signal internal1, internal2 : std_logic;
begin
 -- Instanzen mit Labels BIT0, BIT1, BIT2
 BIT0 : full_adder port map(
   i_bit1 => a_bit0,
   i_bit2 => b_bit0,
   i_carry => i_carry,
   o_sum => o_sum0,
   o_carry => internal1
 );
 [-]
end rtl1;
```

Funktion:



Endliche Automaten

Realisiert durch zwei Prozesse

- SYNC_PROC: Synchroner Prozess → Zustandsregister
  - Bei jeder steigenden Taktflanke übernimmt Current State (CS) den Wert von Next State (NS)

Beispiel

```
SYNC_PROC: process (CLOCK, RESET)
begin
 if (RESET = '1') then
  CS <= S1;
 elsif (CLOCK'event and CLOCK = '1') then
  CS <= NS;
 end if;
end process;
```

- COMB_PROC: Kombinatorischer Prozess → Übergangs- & Ausgabelogik
  - Berechnet aus aktuellem Zustand (CS) und Eingängen sowohl Next State (NS) als auch Ausgänge

Beispiel

```
COMB_PROC : process (CS, A, B, C, D, E) -- Sensitivitätsliste : CS + Inputs
begin
  case CS is
    when S1 =>
      Multi <= '0'; CONTIG <= '0'; SINGLE <= '0'; -- Moore Outputs
      if (A and not B and C) then NS <= S2;
      elif (A and B and not C) then NS <= S4;
      else NS <= S1;
      end if;
    when S2 =>
      MULTI <= '1'; CONTIG <= '0'; SINGLE <= '0';
      if (not D) then NS <= S3; else NS <= S4; end if

    -- ... weitere Zustände
  end case;
end process;
```